

FIT1045 Practice Test

Student name:	Alston Lian Guon You							
Student id:	3	7	4	4	0	9	8	5
Fill circles completely								
Good:								
Bad:								
light fill								
crossing								
part fill								

For this test you:

- must work alone.
- must not use any reference materials.
- will have 60 minutes to complete it.
- must write code in C++ following the standards and conventions of the unit
- must write all answers within this test booklet.
- must hand back your test booklet at the end of the test session.
- must not take any notes or remove any test materials from the test location.
- must place all smart devices away as instructed.
- must not leave in the last 10 minutes of the test.
- should aim to put your answers in the boxes provided (use the space as a guide)
- may use the back of a page if required
- must not fill in, or otherwise write within, the Official Use box below

Official Use (do not fill)		
☐ Complete		☐ Redo

The following criteria will be used to assess this test.

Redo is required if any of the following are present:

- ☐ Any question is left blank
- ☐ Hand execution uses the wrong format
- ☐ Another reason as determined by the assessment panel

Reason:

Redo is required if the submission does not consistently show competency in:

- ☐ Writing code that aligns with, and only with, the given context, requested functionality, and indicated design..
- ☐ Only makes use of the provided procedures, functions, and language features covered in the unit.
- ☐ Correctly tracing the execution of a program by showing creation of, and changes to, values in memory and any terminal outputs.
- ☐ Creation and use of functions and procedures, including correct use of parameters and arguments.
- ☐ Correct use of pass by reference and pass by value.
- ☐ Creation and use of arrays, including indexing, iteration, and population.
- ☐ Creation and use of structs.
- ☐ Correct and appropriate sequence and use of statements to achieve the requested program functionality.
- ☐ Largely correct use of C++ language syntax, and adherence to unit coding conventions.

There may be more issues in your work than shown above. Marking may have stopped once the result of your test became clear (similar to short circuit evaluation of conditions in code).

If you need to redo the test, ensure that you review the entire paper and evaluate your performance against these criteria yourself before booking in for another session. Note that test format and questions will differ between sessions, but these assessment criteria will remain the same.

Q1) Hand execute the following code within the space below. Ensure you follow the process and format described for doing this in the Field Guide.

```
int first[5] = {1, 1, 3, 4, 5};
int first_len = 5;

int second[3] = {2, 2, 3};
int second_len = 3;

int result[5] = {}; // 0 all values
int len = 5;

int i = 0, j = 0, k = 0;
while (i < first_len && j < second_len && k < len)
{
    if (first[i] <= second[j])
    {
        result[k] = first[i];
        i++;
    }
    else
    {
        result[k] = second[j];
        j++;
    }
    k++;
}

while (i < first_len && k < len)
{
    result[k] = first[i];
    k++;
    i++;
}

while (j < second_len && k < len)
{
    result[k] = second[j];
    k++;
    j++;
}
```

first

0	1	2	3	4
1	1	3	4	5

second

0	1	2
2	2	3

result

0	1	2	3	4
1	1	2	2	3

first_len

second_len

len

i

j

k

The following scenario applies to the remaining questions.

Test Details: This program will allow the user to record Test data, where each Test has fields for:

- id (integer), and
- Student (string),
- Score (integer).

The user will interact with this program via the Terminal.

Use the following signatures and types that you may use within your code:

```

o string read_string(string prompt);
o int read_integer(string prompt);
o double read_double(string prompt);
o bool read_boolean(string prompt); //reads Y/N returns true/false
o void write(...);
o void write_line(...);
o string to_string(...);
o dynamic_array<T> with arr[idx], add(arr, value), remove(arr,
index), and length(arr)

```

Here is an example execution of this program (user input in bold):

```

How many Tests to add: 3
Enter ID: 10
Enter Student: Jo
Enter Score: 51

```

```

Enter ID: 20
Enter Student: Kristine
Enter Score: 97

```

```

Enter ID: 152
Enter Student: Jingyu
Enter Score: 83

```

```

Enter id of Test to update: 20
Update Student: n
Update Score: y
Enter Score: 98

```

```

Enter id of Test to delete: 20
Kristine 97
Enter student name to confirm delete: Whoops
Test not deleted

```

```

Tests:
Jo (10) 51
Kristine (20) 98
Jingyu (152) 83
Average: 77.3333333

```

In this program you need to demonstrate effective use of arrays, structs, control flow, function and procedure creation and use, references, and parameters.

Q2) Write the code to declare a struct to store a single **Test**:

```
struct test_data
```

```
{
```

```
    int id;
```

```
    String student;
```

```
    int score;
```

```
};
```

Q3) Write a procedure to **output/write** a single Test to the terminal in the format shown in the example output on page 4. The procedure must accept a Test as a parameter using your struct from Q2.

```
void  
{
```

```
draw_data (test_data test)
```

```
write_line (test.student + " (" + to_string(test.id) + ") " + to_string(test.score));
```

```
}
```

Q4) Write a function to **input/read** a single Test from the user, as shown in the example output on page 4. This function must have no parameters, and return Test data using your struct from Q2.

```
test_data read_data ( )
```

```
{
```

```
test_data test;
```

```
test.id = read_integer("Enter ID: ");
```

```
test.student = read_string("Enter student: ");
```

```
test.score = read_integer("Enter Score: ");
```

```
return test;
```

```
}
```

Q5) In the space below write a function to find a Test in an array. This will be passed 2 parameters:

-) a prompt string
-) a dynamic array of Test values

The function will return an integer.

It will:

- Output the prompt (using write_line)
- Ask the user for the id of the Test to find (using read_integer).
- Search the array, and when a Test with a matching id is found, its index in the array will be returned.
- If the search ends without finding a matching Test, then the function returns -1.

```
int find_test (string prompt, dynamic_array<test_data> test)
{
    write_line (prompt);
    int x = read_integer("Enter a id of test to find: ");
    for (int i=0 ; i < length(test); i++)
    {
        if (test[i].id == x)
        {
            return i;
        }
    }
    return -1;
}
```

- Q6) In the space below write a function to calculate the average Score from an array of Tests (using your struct from Q2). It will be passed the array as a parameter, and return a double.

```
double average (dynamic_array<test_data> test )  
{  
    int sum = 0;  
    for (int i=0; i < length(test); i++)  
    {  
        sum += test[i].score;  
    }  
    return (double) sum / length(test);  
}
```

- Q7) In the space below write a procedure to update Test data. This will be passed 1 parameter:
- o) Test data by reference

It will:

- Ask the user if they want to update the Student - using read_boolean
- If they do, then read in a new value and update the Student
- Ask the user if they want to update the Score - using read_boolean
- If they do, then read in a new value and update the Score

```
void update_test (test_data & test)
```

```
{
```

```
    bool update_b = read_boolean("Update student: ");
```

```
    if (update_b == True)
```

```
    {
```

```
        test.student = read_string("Enter student: ");
```

```
    }
```

```
    update_b = read_boolean("Update score: ");
```

```
    if (update_b == True)
```

```
    {
```

```
        test.score = read_integer("Enter score: ");
```

```
    }
```

```
}
```

Q8) In the space below write a procedure to remove/delete a Test. This will be passed two parameters:

- the dynamic array of Tests (by reference),
- the index of the Test to remove,

It will:

- Check that the index is valid - output a message and return if not
- Output the Student and Score for the Test to be deleted
- Ask the user to enter the Student name to confirm the delete should occur
- Remove the Test from the array if the value entered equals the Student for the Test to be deleted. (use `remove(arr, index)` from `splashkit-array.h`)
- Otherwise, output a message if the value entered does not match the Student for the test to be deleted.

```

void remove_test (dynamic_array<test_data> &test, int index)
{
    if (index < 0 || index >= length(test))
    {
        write_line ("Index is invalid.");
        return ;
    }

    write_line (test[index].student + " " + to_string(test[index].score) );
    string student_name = read_string ("Enter student name to confirm delete: ");
    if (test[index].student == student_name)
    {
        remove (test , index );
    }
    else
    {
        write_line ("Test not deleted");
    }
}

```

- Q9) Implement **main** that will
- o) Declare a dynamic array of Tests using your struct from Q2
 - o) Ask the user how many Tests they want to add.
 - o) Loop to add the required Tests. Each loop will:
 - i) Read a Test in from the user – using your function from Q4
 - ii) Add it to the array
 - o) Call your function from Q5 to find the index of a Test to update.
 - o) If the index is not -1, then update the Test at that index in the array using your function from Q7.
 - o) Call your function from Q5 to find the index of a Test to delete.
 - o) Call your procedure from Q8 to delete the Test.
 - o) Print all of the Tests from your collection – using your procedure from Q3.
 - o) Output the average – calling your function from Q6

```

int main( )
{
    dynamic_array<test_data> test;
    int num_test = read_integer("How many tests want to add: ");
    for (int i=0; i < num_test; i++)
    {
        add (test, read_data());
    }

    int index=find_test("Enter id of Test to update: ", test);
    if (index != -1)
    {
        update_test (test[index]);
    }

    index = find_test("Enter id of Test to delete: ", test);
    remove_test (test, index);
    for (int i=0; i < length(test); i++)
    {
        draw_data (test[i]);
    }
    write_line ("Average: " + to_string(average(test)));
}

```